

**Simon Grundner**  
**k12136610**

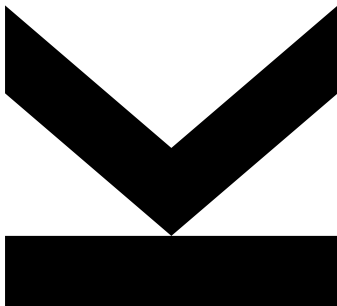
Institute for Integrated  
Circuits and Quantum  
Computing

@ k12136610@students.jku.at

 [https://github.com/s-  
grundner/LVA-EIC-KV](https://github.com/s-grundner/LVA-EIC-KV)

December 23, 2025

# Tiny Tapeout - MIDI Polyphonic Synthesizer



## Technical Report

KV Integrated Circuits Design - WiSe25



**JOHANNES KEPLER  
UNIVERSITY LINZ**  
Altenberger Straße 69  
4040 Linz, Austria  
jku.at

**Abstract** MIDI Polyphonic Synthesizer on the Tiny Tapeout ASIC

## Contents

|                                                    |            |
|----------------------------------------------------|------------|
| <b>I. Abstract</b>                                 | <b>ii</b>  |
| <b>II. Acronyms</b>                                | <b>iii</b> |
| <b>1. Introduction</b>                             | <b>1</b>   |
| <b>2. System Overview</b>                          | <b>1</b>   |
| <b>3. The MIDI Protocol</b>                        | <b>1</b>   |
| 3.1 Twelve Tone Equal Temperament . . . . .        | 1          |
| 3.2 Polyphony . . . . .                            | 2          |
| 3.3 Physical Layer of the MIDI-Protocol . . . . .  | 2          |
| <b>4. Synthesizer</b>                              | <b>2</b>   |
| 4.1 Square-Waveform . . . . .                      | 2          |
| 4.2 Frequency Calculation and Generation . . . . . | 2          |
| 4.3 Oscillatorstack . . . . .                      | 3          |
| <b>5. System Testing</b>                           | <b>3</b>   |
| <b>6. External Hardware</b>                        | <b>3</b>   |
| 6.1 MIDI-Source . . . . .                          | 3          |
| 6.2 Input Side . . . . .                           | 3          |
| 6.3 Output Side . . . . .                          | 4          |
| <b>7. Used Software</b>                            | <b>4</b>   |

## I. Abstract

This project implements a polyphonic audio square wave synthesizer in Verilog, which can be controlled via the Musical Instrument Digital Interface (MIDI) protocol. The project is allocated on the TTSKY25b shuttle of the TinyTapeout project and is to be manufactured under the SkyWater 130nm process. This report documents functionality as well as implementation and discusses design constraints, imposed by the limited physical space on the shuttle.

## II. Acronyms

**MIDI** Musical Instrument Digital Interface

**HDL** Hardware Description Language

**ASIC** Application Specific Integrated Circuit

**PDK** Process Development Kit

## 1. Introduction

MIDI is a versatile digital interface, which packs note expressions into a simple protocol. The note expression often originates from a MIDI-controller (source) and contains information about which key on the pianoroll is affected and how it is affected. A key thing to notice is, that the note expression does not contain any information about how the audio signal itself sounds. The synthesis of the waveform is handled by a MIDI-instrument (sink) and translates the note expression into an audio timesignal. Figure 1 shows how a MIDI-keyboard might interface with a MIDI compatible synthesizer.

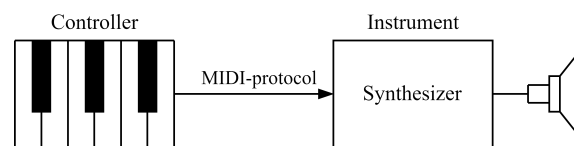


Figure 1: Setup of a controller-instrument-pair communicating via the MIDI-protocol.

The project implements such a MIDI-instrument and converts incoming MIDI-signals into audible squarewaves. The project is written in the Hardware Description Language (HDL) Verilog specifically to be built as a digital Application Specific Integrated Circuit (ASIC) design by the opensource LibreLane toolchain. The resulting chip will be taped out in collaboration with the Tiny-Tapeout project.

The Tiny-Tapeout project makes manufacturing ASIC-designs for individuals feasible by combining multiple smaller designs on a single chip under the usage of an open source Process Development Kit (PDK). The chip space is sectioned in tiles, where each tile has around  $160 \times 100 \mu\text{m}$  of available space. The project on hand occupies one tile. The active run at this time, TTSKY25b, manufactures the chip under SkyWater 130 nm PDK ruleset.

## 2. System Overview

Block Diagram

For this and following sections, backup code with logic diagrams from quartus. Preferably stick to Abstract Block Diagrams made with tikz

Pinout

## 3. The MIDI Protocol

Dataframe

### 3.1 Twelve Tone Equal Temperament

How do Digital Notes Correlate with the Base Frequency.

### 3.2 Polyphony

Overlapping Squarewave in Time and Frequency Domain. Goal is to show, why each voice is output on a individual Pin.

### 3.3 Physical Layer of the MIDI-Protocol

DIN Connector or USB with Differential Pair. No Clock Signal leading to asynchronous Transmission.

Universal Asynchronous Receiver Transmitter

## 4. Synthesizer

How is the Frequency Generated?

Count up a defined Number of steps

For each Count, a time interval of  $1/f_{clk}$  elapses.

The Lowest Frequency requires the highest number of counts

The Task is to (space) efficiently determine the Counter Period from the MIDI-Note number

### 4.1 Square-Waveform

Why? No Wavetable and DAC required. Simple Digital Output

Spectrum:

- For infinite Time: Base Frequency and Harmonics
- For Finite Time: Sincs and Spectral Leakage. Simulation in Ableton or Matlab
- Note Durations can be random. because they are dependent on the user input.

### 4.2 Frequency Calculation and Generation

Limitations:

- Frequency Resolution determined by Clock Frequency - Divisions and Modulo operations are Expensive.

Choosing a Clock Frequency

- See reasoning in Py Notebook

Externalize Calculation Logic from the Oscillator Stack

Compare optimisations

- Performing an actual Twelvetone Temperament Calculation on Hardware
- Lookup Table: Save the Countervalues of the Twelve Notes on the Octave to achieve Most precise Counterperiod. What Calculations are required by the oscillators
- Lookup Table: Reduce Bitwidth and therefor required LUT Size. Compare Size Reduction

Impact of Frequency Deviation

- Systematic error of frequencies from reducing counter resolution. Deviations and calculations from Py-Notebook.
- Spectrum and Harmonics

Are the Deviations Audible?

- Heavily Dependent on the individual Person (Hearing Curve)
- Document Experiment in Ableton Live

### 4.3 Oscillatorstack

Routing a MIDI a Channel to a fixed Oscillator.

Modularity by a Generating Function: Option to increase number of voices for larger space or future Design optimisations.

## 5. System Testing

Edge Cases

Limitations. Which tests are omitted. Correctness of the midi Input Byte Order for example.

CocoTB Testresults, Design Simulation+validation, Measurements, Waveforms

## 6. External Hardware

### 6.1 MIDI-Source

Example of a USB-MIDI controller.

In Principle Direct connection is Possible but a Powersupply is required.

Setting up a Virtual MIDI-Source with a DAW, Hairless MIDI and PMOD as a PHY for the native Differential midi connection.

### 6.2 Input Side

Digital Frontend converting simultaneous Notes into individual Channel

### **6.3 Output Side**

Audio Mixing Circuit, Gain Control, Speaker Driver

Option for mild subtractive Sound-Synthesis

- Squarewaves provide all odd harmonics which can be filtered by an external system to create different Sounddesigns
- optimal Would be a Sawtooth as it contains all harmonics but violated the digitality of the ASIC Output.

## **7. Used Software**